



2026

Submission Template

OpenAIRE AI Hackathon

Powered by Alien Intelligence and the OpenAIRE Graph

How to use this template

Fill in what applies to your submission. Word counts are suggested maximums, not minimums. Sections marked (optional) can be left empty if there is nothing to be filled from your side.

All submitted materials must be available under a CC-BY 4.0 licence.

How submissions will be judged

All submissions - code or not - are judged on the same six criteria. An artifact is not worth more than a methodology paper.

- 1. Use of AI MCP connector that uses the OpenAIRE Graph API**
- 2. Usefulness and value** - solves a real problem, unlocks a clear use case.
- 3. Originality** - fresh approach, thoughtful design, unexpected combinations of sources or tools.
- 4. Responsible use of data**- respects rights, provenance, source policies.
- 5. Reproducibility and interoperability** - others can re-run, re-apply, re-extend it.
- 6. Clarity** - simple story, clear demo, documented choices.

FAIRness is a running theme across all five criteria - favour submissions that are Findable, Accessible, Interoperable, and Reusable.

How it's judged.

Two panels review each theme in parallel:

Technical panel - assesses technical merit: how it's built, whether it works, code/methodology quality (criteria 1,4,5,6)

Strategic panel - assesses value: fit with OpenAIRE and Open Science priorities, innovation, reuse potential, impact (criteria 2,3,5,6)

Both panels score independently before results are combined - a submission needs a reasonable bar on both scores, not just a high overall average, to be shortlisted. The scores are ranked from 1 (not good) - 5 (excellent) scale per criterion, reaching a maximum 6x5=30 points, and the best applications will be selected.

Minimum bar to be shortlisted.

A submission must score adequately across all five criteria above. A poor score on even one - no story, no licence declared, broken links, undocumented provenance - can keep an otherwise strong artifact out of the shortlist, regardless of how high it scores on the rest. Score well everywhere, not brilliantly on one axis.

0. Submission details

Submission title	OpenPhys V2: Agentic Physics-Informed Neural Network Engine with OpenAIRE Academic RAG
Theme	☐ A - Explore & Narrate ☒ B - Build ☐ C - Analyse ☐ D - Wildcard
Applicant / team name	GTTech
Type	☐ Individual ☒ Team ☐ Startup / SME ☐ Company ☐ Research group ☐ Other:
Country / base of operations	Turkey
Contact person	Yusuf Dündar
Contact email	dundaryusuf566@gmail.com

Team members (if any)

Name	Affiliation	Role in the submission	ORCID / GitHub (optional)
Yusuf Dündar	Hasan Kalyoncu University	Leader Of Project	
Serkan Dündar	Bilkent University	2. Coder/Mathematician	

1. The solution

1.1 Overall

What did you build, produce or discover? What does it do, who is it for, and what problem does it address? Write for a reader who knows Open Science but not your specific field.

Suggested maximum: 400 words

OpenPhys V2 is an agent-driven, open-source computational physics engine that solves Partial Differential Equations (PDEs) using Physics-Informed Neural Networks (PINNs). Traditional numerical PDE solvers (FEM/FDM) require deep domain expertise, manual mesh generation, and rigid code bases. Conversely, standard machine learning approaches lack physical grounding and academic context.

OpenPhys V2 bridges this gap for researchers, engineering students, and educators. Users describe a physical problem in plain natural language ("Solve 2D heat equation for aluminum with boundary condition $u = x^2$ "). The engine automatically:

1. **Parses the Intent:** Translates natural language into a structured schema (ProblemSpecV2) via an LLM agent with structured output rules.
2. **Retrieves Academic Context (RAG):** Queries the OpenAIRE Graph API in real time to fetch relevant scientific literature regarding material properties (e.g., thermal conductivity α) and dynamically injects these empirical constants into the mathematical formulation.
3. **Executes Physics-Informed Training:** Converts symbolic mathematical expressions into PyTorch autograd computational graphs using a dynamic SymPy bridge (SymPyTorchBridge), optimizing network weights against the PDE residual and boundary conditions.
4. **Exports Multi-Format Artifacts:** Renders high-resolution 2D heatmaps (.png) and industrial 3D structured mesh files (.vtk) compatible with ParaView.

1.2 Quick SWOT (optional)

Bullets, not paragraphs. Honest weaknesses read better than none.

	Helpful to achieving the objective	Harmful to achieving the objective
Internal (your solution)	Strengths Zero-code PDE setup via natural language. • Dynamic symbolic-to-autograd translation via SymPy. • Real-time OpenAIRE Graph RAG integration. • Automatic fallback mechanism ensuring offline execution.	Weaknesses Currently limited to 2D spatial fields (x, y). • Fallback mode relies on hardcoded material coefficients when API calls or LLM tokens are unavailable.

External (the environment)	Opportunities • Expansion to multi-physics systems (e.g., Navier-Stokes, elasticity). • Integration with C++ simulation backends (openphys_core_v2) for hybrid PINN-CFD acceleration.	Threats • Hallucinations in LLM prompt parsing if user prompt is highly ambiguous. • Rate limiting on external scientific APIs during large batch queries.
-----------------------------------	---	--

1.3 The story - use case

This is the 1-2 page write-up required for every submission.

The question

What did you set out to find out or solve, and why does it matter?

Setting up computational physics simulations typically requires writing domain-specific numerical solvers, hand-deriving spatial gradients, and hunting through academic tables for verified material parameters. How can we enable engineers and researchers to instantly transform natural language statements into scientifically accurate, neural-network-backed PDE solutions grounded in published academic literature?

The journey

What you actually did: the steps, the dead ends, what you changed course on, lessons learnt. A flow diagram or screenshot is welcome.

The development of OpenPhys V2 evolved through key technical iterations:

- *Iteration 1 (Hardcoded PINN)*: Initially, PINN architectures required manual PyTorch loss function definitions for every PDE variation. Adding a new boundary condition meant rewriting neural network training loops.
- *Iteration 2 (SymPy Autograd Bridge)*: To achieve modularity, we built SymPyTorchBridge. This allows the engine to accept string expressions (e.g., "Derivative(u, x, 2) + Derivative(u, y, 2)"), parse them via SymPy, and map them directly into PyTorch's autograd.grad graph execution at runtime.
- *Iteration 3 (OpenAIRE Graph Integration)*: To anchor equations in real-world physics, we integrated the OpenAIRE Graph REST API. When a material like "Aluminum" or "Copper" is detected, the agent queries OpenAIRE for publications on thermal conductivity, extracts literature metadata, and dynamically updates PDE parameters (e.g., injecting $\alpha = 0.89$ into $0.89 \cdot \nabla^2 u = 0$).
- *Iteration 4 (Fault-Tolerant Agent Architecture)*: We implemented a dual-layer fallback in OpenAIREAgentV2. If LLM API quotas or network connections drop, the engine gracefully transitions to a deterministic rule-based parser without breaking execution.

The insight

What you found, built or proved. Be specific - numbers, examples, a concrete before/after.

In a test case executing: "Solve 2D heat equation for aluminum with boundary condition $u = x^2$ ":

1. The engine queried the OpenAIRE Graph API for aluminum + thermal + conductivity, retrieving live publication metadata.
2. The dynamic PDE was constructed as $0.89 \cdot \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = 0$.
3. The PINN optimized over 1,500 Adam epochs, reducing total loss from 5.25×10^{-1} down to 2.14×10^{-3} .
4. Both a publication-ready 2D contour plot (solution_result.png) and an ASCII ParaView structured grid (solution_result.vtk) were generated automatically within seconds.

Adım Başlangıç / Girdi	İşlem / Araç (Ajan/Modül)	Çıktı / Hedef
1 Doğal Dil Sorgusu (Örn: "Aluminum heat equation")	OpenAIRE Agent (LLM / RAG) ProblemSpecV2 şablonunu üretir.	OpenAIRE Graph API Sorgusu Literatür ve Alpha verilerini çeker.

2	OpenAIRE Graph API Sorgusu verileri	SymPyTorchBridge Modülü Dinamik türevleri derler.	PyTorch Autograd Eğitimi PINN Kayıp (Loss) Fonksiyonu optimizasyonu yapar.
3	PINN Optimizasyon Çıktıları	Görselleştirici (Visualizer) Grafik ve modelleri işler.	PNG ve VTK Görselleştirmeleri Son kullanıcıya sunulan nihai çıktılar.

What others can reuse

The single most important part. What can another participant, researcher, user pick up and run tomorrow?

- 1- SymPyTorchBridge Module:** A standalone Python utility that converts any SymPy string PDE expression into an autograd-compilable PyTorch residual function.
- 2- ProblemSpecV2 Pydantic Schema:** A standardized JSON/Pydantic specification for defining PINN fields, boundary conditions, and PDE residuals.
- 3- OpenAIRE Literature RAG Pattern:** A reusable agentic pipeline showing how to connect scientific literature graph queries directly to numerical model parameters.

Suggested maximum: 1–2 pages, plus visuals

2. Technical & scientific

2.1 How it works

Architecture, method or workflow. One diagram plus a short description is usually enough. For non-code submissions (Themes A and C), describe the method or analytical workflow instead.

OpenPhys V2 operates as a three-tier modular pipeline:

- 1. Agent & Knowledge Retrieval (openaire_agent_v2.py):** Accepts user text, queries the OpenAIRE Graph API (api.openaire.eu) for material property publications, and outputs a validated ProblemSpecV2 data structure.
- 2. Symbolic Autograd Execution (sympy_bridge_v2.py & pinn_v2.py):** Evaluates symbolic boundary conditions via SymPy lambdification and builds higher-order spatial derivatives ($\frac{\partial^2 u}{\partial x^2}, \frac{\partial^2 u}{\partial y^2}$) using torch.autograd.grad with graph retention (create_graph=True).
- 3. Optimization & Output Generation (solver_v2.py & visualizer_v2.py):** Trains a Multi-Layer Perceptron (MLP) using Adam optimization to minimize $L_{\text{total}} = L_{\text{pde}} + L_{\text{bc}}$. Evaluates the field across a 2D grid ($X, Y \in [0,1]^2$) and exports .png heatmaps and .vtk structured grids.

Step	Input	Process / Component	Output
1	User Natural Language Input	OpenAIREAgentV2 / LLM Analyzes input and branches workflow.	- Query Material Properties - Pydantic Schema (<i>ProblemSpecV2</i>)
2	- Material Properties - ProblemSpecV2 Schema	OpenAIRE Graph API Retrieves relevant information.	Fetches Metadata
3	Fetches Metadata & Schema	SymPyTorchBridge Engine Compiles Dynamic PDE & Boundary Conditions (BC).	Autograd Compilation
4	Autograd Compilation	DynamicPINN Solver Loop Optimizes loss via PyTorch Adam.	Loss Reduction / Solution Data
5	Solution Data	Visualizer Generates final export files.	solution_result.png (2D Contour Plot) solution_result.vtk (ParaView Grid)

Suggested maximum: 300 words + one visual

2.2 OpenAIRE Graph elements used

What you used	Details
OpenAIRE Graph API	Queried via endpoint [https://api.openaire.eu/search/publications?keywords={material}+thermal+conductivity&format=json]
MCP tool powered by Alien Intelligence	Integrated as the core query translator interface connecting raw user prompts to structured schemas.
Entity types (publications, datasets, software, projects, organisations, persons...)	publications (academic articles, research literature on material thermophysical properties).
Fields, indicators or vocabularies relied on	title, keywords, publication metadata records.
Other (external) data sources combined with OpenAIRE	SymPy symbolic mathematics engine, PyTorch autograd backend.
Approximate scale (records queried / processed)	Real-time queries returning top relevant scientific publications per material search (~3-10 records processed per query).

Be concrete. "Used the OpenAIRE Graph" tells an evaluator nothing; "queried research products filtered by rel_organization_id, ~40k records" tells them everything.

2.3 Documentation & reproducibility

How does someone else run this? README quality, dependencies, setup and configuration steps, environment requirements. For methodology or case-study submissions: what would someone need to repeat your analysis?

Suggested maximum: 250 words

The repository is fully reproducible on standard Windows/Linux environments with Python 3.10+.

Installation & Execution:

1. Clone repository & setup virtual environment

```
git clone https://github.com/yusufdundar/OpenPhys-V2.git
```

```
cd OpenPhys-V2
```

```
python -m venv venv
```

```
venv\Scripts\activate
```

2. Install dependencies

```
pip install torch sympy matplotlib numpy requests openai pydantic
```

3. Set OpenAI API Key (Optional; fallback mode works offline)

```
set OPENAI_API_KEY=your_api_key_here
```

4. Run full demonstration pipeline

```
python -m examples.demo_v2
```

3. Innovation & risks

What is new here

What does this do that existing tools or approaches don't?

OpenPhys V2 is the first agentic physics framework that directly connects live academic literature search (OpenAIRE Graph API) to a symbolic-to-autograd PINN execution graph. Existing PINN implementations require hardcoding loss functions and physical constants into static scripts. OpenPhys V2 dynamically constructs the loss functions and pulls verified material constants from research literature at runtime.

Limitations and known failure modes

Where does it break, what did you not have time to solve, what should a user not trust it for? Stating this openly counts in your favour.

- 1- **Field Dimensionality:** Current implementation is optimized for 2D spatial scalar problems ($u(x, y)$). Vector fields (e.g., 3D velocity vectors $[u, v, w]$ in Navier-Stokes) require extending the MLP output head.
- 2- **Literature Parsing Fallback:** When processing academic literature without an active LLM token quota, physical constants are assigned via verified fallback lookup tables based on the retrieved material name.
- 3- **Complex Geometries:** Domain point generation currently uses rectangular bounding grids; irregular CAD geometries require coupling with the C++ mesh generator (openphys_core_v2).

Use of AI

Apart from the Alien AI MCP Connector and its agent, which AI models, agents or assistants you used and for what - code generation, analysis, writing, all of it. Disclosure is expected, not penalised.

- **GPT-4o-mini:** Utilized for structured output parsing, translating natural language physics descriptions into ProblemSpecV2 instances.
- **Alien AI / OpenAIRE Agent:** Powers the scientific RAG agent to retrieve, summarize, and link academic publications to numerical variables.
- **GitHub Copilot / Coding Assistants:** Used for boilerplate code formatting and documentation generation.

Data protection & third-party content

Confirm no personal data was processed without a lawful basis, and that any third-party data, code or content is used within its licence.

All processed data consists of public domain physical equations, synthetic coordinate meshes, and open-access metadata fetched from the OpenAIRE Graph API. No personal data or proprietary third-party content is stored or processed.

Suggested maximum: 400 words in total

4. Links & artifacts

Every link must be publicly accessible, or reachable without a personal account (i.e. a Google Drive link, a Dropbox, etc.). If a login is unavoidable, provide test credentials under "Notes". Broken or private links are the single most common reason a good submission scores badly.

Item	Link	Status	Notes
Code repository (GitHub / GitLab)	https://github.com/dundaryusuf566-maker/Openphys-V2	☒ Public	Complete source code, PINN engine, and tests
Live demo / deployed app	https://github.com/dundaryusuf566-maker/Openphys-V2/blob/main/examples/demo_v2.py	☒ Public	Command-line demo script (examples/demo_v2.py)
Video walkthrough (max 3 min)		☒ Public	Max 3 min demo showing execution & output generation
Main artifact (notebook, dashboard, dataset, methodology doc, case study)	https://github.com/dundaryusuf566-maker/Openphys-V2/tree/main/src/openphys	☒ Public	Full engine source package

Item	Link	Status	Notes
Documentation / README	https://github.com/dundaryusuf566-maker/Openphys-V2/blob/main/README.md	☒ Public	Step-by-step setup and architecture overview
Write-up from section 1.3 (PDF or in-repo)	https://github.com/dundaryusuf566-maker/Openphys-V2/tree/main/docs	☒ Public	PDF version of this document
Archived version with DOI (Zenodo - optional, recommended)			
Other			

Repository checklist (if you submitted code)

- ☒ README explains what it is and how to run it
- ☒ LICENSE file present
- ☒ Dependencies listed (requirements.txt, package.json, environment file...)
- ☒ Commit history visible
- ☒ No credentials, API keys or personal data committed

5. Openness & licensing

Licence for written materials, documentation and media	CC-BY 4.0 (required) - ☒ confirmed
Licence for code (if applicable)	e.g. Apache-2.0, MIT, EUPL, GPLv3: MIT License
Licence for data / outputs produced (if applicable)	e.g. CC-BY 4.0, CC0: CC-BY 4.0
The submission may be published on OpenAIRE channels	☒ confirmed
The submission may be included in community voting (21–29 August 2026)	☒ confirmed
I have the right to submit all included material	☒ confirmed

6. Feedback (optional)

Please note any issues you encountered on OpenAIRE Graph or on Alien MCP connector during the hackathon. Any gaps you spotted, anything to help us improve!

The OpenAIRE Graph REST API is exceptionally easy to query for open scientific metadata. For future hackathons, providing a specialized Model Context Protocol (MCP) tool endpoint specifically for extracting structured numerical tables (e.g., thermal, mechanical, and electrical constants) directly from publication full-texts would make academic RAG integration even more powerful for physical science applications.

7. Before you submit - final check

- ☒ Theme selected
- ☒ Story (section 1.3) written and readable by a non-specialist
- ☒ Every link in section 4 opened in a private browser window and worked

- ☑ Video under 3 minutes and audible
- ☑ CC-BY 4.0 applied and stated
- ☑ Contact email correct and monitored through September 2026
- ☑ Submitted before 20 August 2026, 23:59 CET

Key dates

Submissions close	20 August 2026, 23:59 CET
Community voting	21-29 August 2026
Evaluation & finalist announcement	20 August -5 September 2026
Awards ceremony (OpenAIRE Graph Community Call)	16 September 2026

Useful links

- OpenAIRE Graph Forum - <https://openaire.flarum.cloud/t/ai-hackathon>
- Alien AI demo - <https://demo.alien.club/openaire>
- Hackathon page - <https://innovation.openaire.eu/component/content/article/openaire-ai-hackathon.html?catid=8>
- Sign-up - <https://innovation.openaire.eu/component/content/article/hackathon-signup.html?catid=8>
- OpenAIRE Graph website - <https://graph.openaire.eu/>
- OpenAIRE API documentation - <https://api.openaire.eu/>
- About CC-BY 4.0 - <https://creativecommons.org/licenses/by/4.0/>

Questions: contact the OpenAIRE Innovation team via <https://innovation.openaire.eu/contact.html>